

Final Exam Review

CSCI 1101

2024-05-07

Exam Format and Length

The final exam will have the same structure as the two midterms.

- written on paper,
- one card of notes allowed,
- about 1 1/2 times as long as a midterm exam.

Exam Structure

Question styles will be familiar:

- true/false and multiple choice
- code reading (what does this function/block return/print?)
- write a function that does...
- write a class that has...

Exam Coverage

All topics covered in the course are fair game, except:

- turtle graphics
- software testing
- CSV file parsing
- python's regular expressions library. But reading and interpreting basic regular expressions (e.g. alternation, iteration, character classes) *is* included.
- the *last lecture* on functional programming (generators and continuations). But the other material on functional programming (e.g. `lambda`, `map`, `filter`, `curry`, `compose`) *is* included.

Exam Emphasis

Most of the questions will be about topics new since the second midterm, specifically:

- exception handling
- defining classes and methods
- class inheritance
- recursive functions
- higher-order functions
- regular expressions (reading only)

Basic Skills

Make sure that you know the basics:

- variable assignment, update and use
- types of literal expressions (e.g. `[1, 2, 3]`, `{1: True, 2: False}`)
- basic operators (e.g. `2 * 2`, `'2' * 2`, `2 < 3`, `{2} < {2, 3}`)
- basic functions, their types, and whether they change the state of their arguments (e.g. `sorted` vs `list.sort`)
- boolean expressions (`n%2 == 0` or `n<10`) and conditional statements and expressions

Loops

Make sure that you are handy with loops:

- `for` vs `while`
- using functions like `range`, `enumerate`, and `zip` with `for` loops
- the accumulator pattern (including mapping and filtering) of iterating over a list

Exceptions

Make sure that you know how to:

- contain statements that might raise an exception in a `try` clause
- handle exceptions that may arise with `except` clauses
- distinguish between exceptions by their type to handle them differently if needed

You do not need to memorize the various exception types, they will be provided in the question, as needed.

Defining Classes

You can expect defining classes to be an important component of the exam:

- syntax for defining classes
- syntax for defining methods (don't forget the `self` parameter!)
- syntax for initializer method (`__init__`)
- syntax for stringification method (`__str__`)
- difference between a *global variable* of a program, *local variable* of a function, *instance attribute* (a.k.a. field) of an object, and *static attribute* of a class.

Class Inheritance

You can expect to encounter class inheritance the exam.

- syntax for defining derived classes (a.k.a. subclasses)
- when and how to override base class methods
- how to call base class methods (`super().method_name(...)`)

Recursion

You can expect to encounter questions about recursive functions.

- what makes a function recursive
- what is a base case
- what is a recursive call
- how to translate a recursive specification into a recursive python function

Function Expressions and Higher-Order Functions

You will be expected to be able to read and write function expressions and functions that take other functions as arguments or return them as results.

- read and write a function expression (a.k.a. `lambda`)
- distinguish between calling and referring to a function
- know how to pass a function as an argument
- know how to call a function returned by another function
- use the built in `map` and `filter` functions that turn accumulator pattern instances into functions

Practice Writing Functions

Write a function that returns the sum of the *positive* numbers in its argument list (or 0 if the list is empty).

For example:

```
> sum_pos([1, -2, 3, -4, 5])  
9
```

- using loop iteration
- using recursion
- using higher-order functions

Practice Writing Classes

Write a class for a `Lamp` which has a boolean state for on/off, and methods to:

- initialize a new lamp with a given name to the off state (`__init__`),
- display the name and state of the lamp (`__str__`),
- toggle the state of the lamp (`toggle`).

Extend the `Lamp` class to a `DimmingLamp` which additionally had a brightness between `0.0` and `1.0` (the default) and a method to set it (`dim`).

To Do

Finish *Project 4: Bioinformatics* on CodeRunner in by 2024-05-08 (no extensions).